

3. 사용자 권한 - 토르망치도 권한을 부여하면 들 수 있을까

[MySQL Setup](#)

3. 사용자 권한

💡 MySQL의 사용자 계정을 생성하는 방법이나 계정의 권한을 설정하는 방법은 다른 DBMS와는 조금 차이가 있다. 대표적으로 MySQL의 사용자 계정은 단순히 사용자의 아이디뿐 아니라 해당 사용자가 어느 IP에서 접속하고 있는지도 확인한다. MySQL 8.0부터 권한을 묶어서 관리하는 역할의 개념이 도입됐기 때문에 각 사용자의 권한으로 미리 준비된 권한 세트를 부여하는 것도 가능하다.

📌 3.1 사용자 식별

MySQL의 사용자는 다른 DBMS와는 조금 다르게 사용자의 계정뿐 아니라 사용자의 접속 지점도 계정의 일부가 된다.

접속 지점: 클라이언트가 실행된 호스트명이나 도메인 또는 IP 주소

따라서 MySQL에서 계정을 언급할 때는 항상 **아이디와 호스트를 함께 명시**해야 한다. 아이디와 IP 주소를 감싸는 따옴표(')는 MySQL에서 식별자를 감싸는 따옴표 역할을 한다. 종종 홑따옴표(')로 바뀌어서 사용되기도 한다.

예시

다음 계정은 MySQL 서버가 기동 중인 로컬 호스트에서 test_id라는 아이디로 접속할 때만 사용될 수 있는 계정이다. 만약 사용자 계정에 다음과 같은 계정만 등록되어 있다면 다른 컴퓨터에서는 test_id라는 아이디로 접속할 수 없음을 의미한다.

```
'test_id'@'127.0.0.1'
```

만약 모든 외부 컴퓨터에서 접속 가능한 사용자 계정을 등록하고 싶다면? 호스트 부분을 % 문자로 대체한다. %는 모든 IP 또는 모든 호스트명을 의미한다.

⚠️ 주의할 점

서로 동일한 아이디가 있을 때, MySQL 서버가 해당 사용자의 인증을 위해 어떤 계정을 선택할까?

예시

```
'test_id'@'192.168.0.10' (pw: 123)
'test_id'@'%' (pw: abc)
```

권한이나 계정 정보에 대해 MySQL은 범위가 가장 작은 것을 항상 먼저 선택한다. 두 계정 정보 가운데 범위가 좁은 것은 %가 포함되지 않은 'test_id'@'192.168.0.10' 이기 때문에 IP가 명시된 계정 정보를 이용해 사용자를 인증하게 된다. 결국 중첩된 계정을 생성하지 않는 것이 제일 좋다.

📌 3.2 사용자 계정 관리

◆ 3.2.1 시스템 계정과 일반 계정

MySQL 8.0부터 **SYSTEM_USER** 권한을 가지고 있느냐에 따라 시스템 계정과 일반 계정으로 구분된다.

1) 시스템 계정

- DB Server 관리자를 위한 계정이다.
- 일반 계정 및 시스템 계정을 관리한다. (생성, 수정, 삭제)
- DB Server 관리와 중요 작업을 담당한다.

1.1) 중요 작업

- 계정 관리 (계정 생성, 삭제, 권한 부여 및 제거 등)
- 다른 세션 또는 그 세션에서 실행 중인 쿼리 강제 종료
- 스토어드 프로그램 생성 시 DEFINER를 타 사용자로 설정

📁 스토어드 프로그램

- 프로시저, 함수, 트리거, 이벤트 같은 것을 말한다.
- DB 안에서 프로그램처럼 저장되어서 명령어 하나로 복잡한 작업을 자동으로 실행할 수 있다.

📁 DEFINER

- 이 스토어드 프로그램을 실행할 때 누구의 권한으로 실행할지 정하는 것이다.
- DEFINER = 'user_name'@'host' → 설정 방법: 이 프로그램을 실행하는 사용자는 누구의 권한으로 실행할 것인가?

2) 일반 계정

- 응용 프로그램 또는 개발자를 위한 계정이다.
- 시스템 계정 관리나 다른 사용자 계정에 대한 제어가 불가능하다.

이렇게 시스템 계정과 일반 계정의 개념이 도입된 것은 DBA 계정에는 SYSTEM_USER 권한을 할당하고 일반 사용자를 위한 계정에는 SYSTEM_USER 권한을 부여하지 않게 하기 위해서이다.

참고

- 사용자: MySQL 서버를 사용하는 주체 (사람 또는 응용프로그램)

- 계정: MySQL 서버에 로그인하기 위한 식별자 (로그인 아이디)

◆ MySQL 내장 계정

MySQL 서버에는 root를 제외한 다음과 같은 내장된 계정들이 있다. 이 계정은 내부적으로 각기 다른 목적으로 사용되므로 삭제되지 않도록 주의해야 한다.

계정명	용도
'mysql.sys'@'localhost'	MySQL 8.0부터 기본으로 내장된 sys 스키마의 객체들의 DEFINER로 사용되는 계정
'mysql.session'@'localhost'	MySQL 플러그인이 서버로 접근할 때 사용되는 계정
'mysql.infoschema'@'localhost'	information_schema에 정의된 뷰의 DEFINER로 사용되는 계정

⚠ 이 계정들은 처음부터 잠겨있는 상태이다.

◆ 3.2.2 계정 생성

MySQL 5.7까지는 GRANT 명령으로 권한의 부여와 동시에 계정 생성이 가능했다. 하지만 **8.0부터 CREATE USER 명령과 GRANT 명령으로 구분해서 실행하도록 바뀌었다.**

계정을 생성할 때 다양한 옵션을 설정할 수 있다.

- 계정의 인증 방식과 비밀번호
- 비밀번호 관련 옵션: 유효기간, 이력 개수, 재사용 불가 기간
- 기본 역할
- SSL 옵션
- 계정 잠금 여부

예시

```
CREATE USER 'user'@'%'
  IDENTIFIED WITH 'mysql_native_password' BY 'password' -- 사용자 인증 방식과
  비밀번호 설정
  REQUIRE NONE -- 암호화된 SSL/TLS 채널
  사용 여부
  PASSWORD EXPIRE INTERVAL 30 DAY -- 비밀번호 유효기간 설정
  옵션
  ACCOUNT UNLOCK -- 계정 잠금 여부
  PASSWORD HISTORY DEFAULT -- 한번 사용한 비밀번호
  재사용 못하게 설정하는 옵션
  PASSWORD REUSE INTERVAL DEFAULT -- 한번 사용한 비밀번호
  재사용 금지 기간 설정
  PASSWORD REQUIRE CURRENT DEFAULT; -- 새로운 비밀번호로 변경
  할 때 현재 비밀번호를 필요로 할지 말지 결정
```

3.2.2.1 IDENTIFIED WITH

사용자의 인증 방식과 비밀번호를 설정한다. IDENTIFIED WITH 뒤에 반드시 인증 방식을 명시해야 한다. 기존 인증 방식이면 IDENTIFIED BY 'password' 형식이다.

MySQL 서버에서는 다양한 인증 방식을 플러그인 형태로 제공하며, 다음 4가지 방식이 가장 대표적이다.

인증 방식

- Native Pluggable Authentication
- Caching SHA-2 Pluggable Authentication
- PAM Pluggable Authentication
- LDAP Pluggable Authentication

1) Native

- MySQL 5.7까지 사용된 방식이다.
- 비밀번호에 대한 SHA-1 알고리즘의 해시값을 저장하고 클라이언트가 보낸 값과 해시값이 일치하는지 비교 인증한다.

2) Caching SHA-2

- MySQL 8.0 버전에서 보완된 인증 방식이다.
- SHA-2 (256비트) 알고리즘을 사용한다.
- 저장된 해시값의 보안에 더 중점을 두었다.
- 내부적으로 Salt 키가 필요하며 수천 번의 해시 계산을 수행해서 결과를 생성한다. 따라서 동일한 키값에 대해서도 다른 결과가 나온다.
- 하지만 시간 소모적이고 성능이 저하될 수 있다.
- 이를 보완한 것이 해시 결과값을 메모리에 캐시해서 사용하는 것이다.
- 이 인증 방식을 사용하려면 SSL/TLS 또는 RSA 키페어를 반드시 사용해야 한다.
- 클라이언트에서 접속 시 SSL 옵션을 활성화해야 한다.

3) PAM

- 유닉스나 리눅스 패스워드 또는 LDAP 같은 외부 인증을 사용할 수 있게 해주는 인증 방식이다.
- 엔터프라이즈 에디션에서만 사용 가능하다.

4) LDAP

- LDAP를 이용한 외부 인증을 사용할 수 있게 해주는 방식이다.
- 엔터프라이즈 에디션에서만 가능하다.

MySQL 5.7까지는 Native Authentication이 기본 인증 방식이지만, MySQL 8.0부터 Caching SHA-2 Authentication이 기본 인증이다. 만약 보안 수준은 낮지만 기존 버전과의 호환성을 고려한다면 Caching SHA-2보다는 Native 인증 방식으로 계정을 생성할 수 있다.

MySQL 8.0에서도 Native를 기본 인증 방식으로 설정이 가능하다.

- my.cnf 설정 파일에 추가하면 된다.
 - `SET GLOBAL default_authentication_plugin = "mysql_native_password"`
-

3.2.2.2 REQUIRE

MySQL 서버에 접속할 때 암호화된 SSL/TLS 채널을 사용할지 여부를 설정한다. 별도로 설정하지 않으면 비암호화 채널로 연결한다.

만약 Caching SHA-2 Authentication 인증 방식을 사용하면 암호화된 채널만 사용 가능하다.

3.2.2.3 PASSWORD EXPIRE

비밀번호의 유효기간을 설정하는 옵션이다. 별도로 명시하지 않으면 `default_password_lifetime` 시스템 변수에 저장된 기간으로 유효기간이 설정된다.

개발자 또는 DBA의 비밀번호는 유효기간을 설정하는 것이 보안상 안전하지만, 응용 프로그램 접속용 계정에 유효기간을 설정하는 것은 위험할 수 있다.

왜 위험한가?

- 유효기간이 만료되면 응용 프로그램이 더 이상 DB에 접속할 수 없다.
- 웹사이트나 서비스가 장애 상태가 되어 사용자에게 서비스 불가 오류가 발생한다.
 - 예: 사이트가 잘 돌아가다가 DB 접속 오류로 다운되는 현상
- DB 계정의 유효기간을 자동으로 갱신할 수 없다.
 - 관리자가 직접 연장해야 하는데, 이를 깜빡하면 장애 발생 가능성이 올라간다.

PASSWORD EXPIRE 옵션

- `PASSWORD EXPIRE` : 계정 생성과 동시에 비밀번호의 만료 처리
 - `PASSWORD EXPIRE NEVER` : 만료 기간 없음 (웹/API 계정에 권장)
 - `PASSWORD EXPIRE DEFAULT` : `default_password_lifetime` 시스템 변수에 저장된 기간으로 비밀번호의 유효기간을 설정한다.
 - `PASSWORD EXPIRE INTERVAL n DAY` : 비밀번호의 유효기간을 오늘부터 n일자로 설정
-

3.2.2.4 PASSWORD HISTORY

한 번 사용했던 비밀번호를 재사용하지 못하게 설정하는 옵션이다. `PASSWORD HISTORY` 절에 설정 가능한 옵션은 다음과 같다.

- `PASSWORD HISTORY DEFAULT`
 - `password_history` 시스템 변수에 저장된 개수만큼 비밀번호의 이력을 저장하며, 저장된 이력에 남아있는 비밀번호는 재사용할 수 없다.
- `PASSWORD HISTORY n`

- 비밀번호의 이력을 최근 n개까지만 저장한다.
- 저장된 이력에 남아있는 비밀번호는 재사용할 수 없다.

한 번 사용한 비밀번호를 사용하지 못하게 하려면 이전에 사용했던 비밀번호를 MySQL 서버가 기억해야 한다. 이를 위해 MySQL 서버는 mysql DB의 password_history 테이블을 사용한다.

3.2.2.5 PASSWORD REUSE INTERVAL

한 번 사용한 비밀번호의 재사용 금지 기간을 설정하는 옵션이다. 별도로 명시하지 않으면 password_reuse_interval 시스템 변수에 저장된 기간으로 설정된다.

설정 가능한 옵션은 다음과 같다.

- PASSWORD REUSE INTERVAL DEFAULT
 - password_reuse_interval 변수에 저장된 기간으로 설정
 - PASSWORD REUSE INTERVAL n DAY
 - n일자 이후에 비밀번호를 재사용할 수 있게 설정
-

3.2.2.6 PASSWORD REQUIRE

비밀번호가 만료되어 새로운 비밀번호로 변경할 때 현재 비밀번호(변경하기 전 만료된 비밀번호)를 필요로 할지 말지를 결정한다. 별도로 명시하지 않으면 password_require_current 시스템 변수의 값으로 설정된다.

설정 가능한 옵션은 다음과 같다.

- CURRENT
 - 비밀번호를 변경할 때 현재 비밀번호를 먼저 입력하도록 설정
 - OPTIONAL
 - 비밀번호를 변경할 때 현재 비밀번호를 입력하지 않아도 되도록 설정
 - DEFAULT
 - password_require_current 시스템 변수의 값으로 설정
-

3.2.2.7 ACCOUNT LOCK/UNLOCK

계정 생성 시 또는 ALTER USER 명령을 사용해 계정 정보를 변경할 때 계정을 사용하지 못하게 잠글지 여부를 결정한다. 계정 자체는 존재하지만 로그인을 못하게 할 수 있는 기능이다.

- ACCOUNT LOCK : 계정을 사용하지 못하게 잠금
 - ACCOUNT UNLOCK : 잠긴 계정을 다시 사용 가능한 상태로 잠금 해제
-

📌 3.3 비밀번호 관리

◆ 3.3.1 고수준 비밀번호

MySQL 서버의 비밀번호는 유효기간이나 이력 관리를 통한 재사용 금지 기능뿐만 아니라 비밀번호를 쉽게 유추할 수 있는 단어들이 사용되지 않게 글자의 조합을 강제하거나 금칙어를 설정하는 기능도 있다. 유효성 체크 규칙을 적용하려면 `validate_password` 컴포넌트를 이용해야 한다.

```
INSTALL COMPONENT 'file://component_validate_password';
```

비밀번호 정책에는 3가지가 있으며 이 중 하나를 선택할 수 있다. 기본값은 MEDIUM으로 자동 설정된다.

비밀번호 정책 레벨

LOW

- 비밀번호의 길이만 검증한다.

MEDIUM

- 비밀번호의 길이를 검증하며, 숫자와 대소문자 그리고 특수문자의 배합을 검증한다.

STRONG

- MEDIUM 레벨의 검증을 모두 수행하며, 금칙어가 포함됐는지 여부까지 검증한다.

비밀번호의 길이는 `validate_password.length` 시스템 변수에 설정된 길이 이상의 비밀번호가 사용됐는지 검증하며, 숫자와 대소문자, 특수문자는 `validate_password.mixed_case_count`와 `validate_password.number_count`, `validate_password.special_char_count` 시스템 변수에 설정된 글자 수 이상을 포함하고 있는지 검증한다. 금칙어는 `validate_password.dictionary_file` 시스템 변수에 설정된 사전 파일에 명시된 단어를 포함하고 있는지를 검증한다.

금칙어 파일은 금칙어들을 한 줄에 하나씩 기록해서 텍스트 파일로 작성하면 된다. 그리고 MySQL 서버에 금칙어 파일을 등록한다. 단, 금칙어 관련 설정은 비밀번호 정책이 STRONG인 경우에만 작동한다.

```
SET GLOBAL validate_password.dictionary_file='prohibitive_word.data';  
SET GLOBAL validate_password.policy='STRONG';
```

◆ 플러그인 vs 컴포넌트

플러그인

- 기존의 MySQL 서버에 특정 기능을 플러그인처럼 추가 및 제거하는 구조이다.
- `INSTALL PLUGIN`, `UNINSTALL PLUGIN` 을 사용한다.
- 설치 위치는 `plugin` 디렉토리에 `.so`, `.dll` 파일이 존재한다.

장점

- 비교적 간단한 설치 및 제거

- 안정적

단점

- 독립성이 낮다
- 내부 API 접근이 제한적
- 보안 및 확장성 면에서 부족하다

컴포넌트

- MySQL 8.0부터 새롭게 도입되었다.
- 모듈화된 확장 구조
- 서버 기능을 더 유연하게 확장하고 통합하기 위한 구조이다.

```
INSTALL COMPONENT 'file://component_validate_password';
```

장점

- 기능 간 의존성 관리와 인터페이스가 명확하여 모듈화가 우수하다.

단점

- 구조가 복잡하다.
- 설정/관리의 학습 곡선이 존재한다.

◆ 3.3.2 이중 비밀번호

일반적으로 많은 응용 프로그램 서버들이 공용으로 데이터베이스 서버를 사용하기 때문에 데이터베이스 서버의 계정 정보는 응용 프로그램 서버로부터 공용으로 사용되는 경우가 많다. 이러한 특성으로 인해 데이터베이스 서버의 계정 정보는 쉽게 변경하기 어려운데, 그중에서도 데이터베이스 계정의 비밀번호는 서비스가 실행 중인 상태에서 변경이 불가능했다. 그래서 처음 설정한 상태로 몇 년 동안 사용되는 경우가 많다. 하지만 서비스를 모두 멈추지 않고서는 비밀번호를 변경하는 것은 불가능한 일이었다. 이러한 문제점을 해결하기 위해 MySQL 8.0부터 계정의 비밀번호로 2개의 값을 동시에 사용할 수 있는 기능을 추가했다.

여기서 이중 비밀번호는 2개의 비밀번호 중 하나만 일치하면 로그인이 통과되는 것을 의미한다. 영어로는 Dual Password이다.

2개의 비밀번호는 Primary와 Secondary로 나뉜다. 최근에 설정된 비밀번호가 Primary 비밀번호이며, 이전 비밀번호는 Secondary 비밀번호가 된다. 이중 비밀번호를 사용하려면 기존 비밀번호 변경 구문에 `RETAIN CURRENT PASSWORD` 옵션만 추가하면 된다.

예시

1. 비밀번호를 1234로 설정
2. 비밀번호를 4321로 설정 → 이때 옵션으로 `RETAIN CURRENT PASSWORD` 추가

순서

1. 처음에는 Primary 비밀번호가 1234이며 Secondary는 빈 상태이다.
2. 그다음 4321이 Primary 비밀번호가 된다.
3. 1234가 Secondary 비밀번호가 된다.

이 둘 중 아무거나 입력해도 로그인이 가능하다.

추후 보안을 위해 Secondary 비밀번호는 삭제하는 것이 좋다.

```
ALTER USER 'root'@'localhost' DISCARD OLD PASSWORD;
```

이렇게 Secondary 비밀번호가 삭제되면 기존 비밀번호로는 로그인이 불가능하다. 새로운 Primary 비밀번호만 로그인할 수 있게 된다.

3.4 권한(Privilege)

MySQL 5.7 버전까지 권한은 글로벌(Global) 권한과 객체 단위의 권한으로 구분됐다. 데이터베이스나 테이블 이외의 객체에 적용되는 권한을 글로벌 권한이라고 하며, 데이터베이스나 테이블을 제어하는 데 필요한 권한을 객체 권한이라고 한다. 객체 권한은 GRANT 명령으로 권한을 부여할 때 반드시 특정 객체를 명시해야 하며, 글로벌 권한은 GRANT 명령에서 특정 객체를 명시하지 말아야 한다.

예를 들어 특정 객체에 ALL 권한이 부여되면 해당 객체에 적용될 수 있는 모든 객체 권한을 부여하며, 글로벌로 ALL이 사용되면 글로벌 수준에서 가능한 모든 권한을 부여하게 된다.

글로벌 권한

- DB나 테이블 이외의 객체에 적용되는 권한
- GRANT 명령 시 특정 객체를 명시하지 않는다.

객체 권한

- DB나 테이블 등을 제어하는 데 필요한 권한
- GRANT 명령 시 특정 객체를 명시해야 한다.
- 더 세부적인 단위로 권한을 제한할 수 있다.

```
MySQL Server
├─ [글로벌 권한]
├─ [사용자]
└─ mydb (객체 권한)
    ├── table1
    └── table2
```

MySQL 8.0부터 동적 권한이 추가됐다. 정적 권한은 MySQL 서버의 소스코드에 고정적으로 명시돼 있는 권한을 의미하며, 동적 권한은 MySQL 서버가 시작되면서 동적으로 생성하는 권한을 의미한다. 예를 들어, MySQL 서버의 컴포넌트나 플러그인이 설치되면 그때 등록되는 권한을 동적 권한이라고 한다.

MySQL 5.7까지 SUPER라는 권한이 데이터베이스 관리를 위해 꼭 필요한 권한이었지만, MySQL 8.0부터는 SUPER 권한은 잘게 쪼개어져 동적 권한으로 분산됐다. 그래서 MySQL 8.0부터 백업 관리자와 복제 관

리자 개별로 꼭 필요한 권한만 부여할 수 있게 된 것이다.

권한 부여 문법

권한을 부여할 때는 GRANT 명령을 사용한다. GRANT 명령은 다음과 같은 문법으로 작성하는데, 각 권한의 특성(범위)에 따라 GRANT 명령의 ON 절에 명시되는 오브젝트(DB나 테이블)의 내용이 바뀌어야 한다.

```
GRANT privilege_list ON db.table TO 'user'@'host';
```

8.0부터는 존재하지 않는 사용자에게 GRANT 명령이 실행되면 에러가 발생하므로 사용자를 먼저 생성하고 GRANT 명령으로 권한을 부여해야 한다. GRANT OPTION 권한은 다른 권한과는 달리 GRANT 명령의 마지막에 WITH GRANT OPTION을 명시해서 부여한다. privilege_list에는 구분자(,)를 써서 권한 여러 개를 동시에 명시할 수 있다. TO 키워드 뒤에는 권한을 부여할 대상 사용자를 명시하고, ON 키워드 뒤에는 어떤 DB의 어떤 오브젝트에 권한을 부여할지 결정할 수 있다.

글로벌 권한

```
GRANT SUPER ON *.* TO 'user'@'localhost';
```

글로벌 권한은 특정 DB나 테이블에 부여될 수 없기 때문에 글로벌 권한을 부여할 때 GRANT 명령의 ON 절에는 항상 *.* 을 사용하게 된다. *.* 은 모든 DB의 모든 오브젝트를 포함해서 MySQL 서버 전체를 의미한다.

DB 권한

```
GRANT EVENT ON *.* TO 'user'@'localhost';
GRANT EVENT ON employees.* TO 'user'@'localhost';
```

DB 권한은 특정 DB에 대해서만 권한을 부여하거나 서버에 존재하는 모든 DB에 대해 권한을 부여할 수 있기 때문에 위의 예제와 같이 ON 절에 *.* 이나 employees.* 모두 사용할 수 있다. 여기서 DB라고 하면 DB 내부에 존재하는 테이블뿐만 아니라 스토어드 프로그램들도 모두 포함한다.

DB 권한은 데이터베이스 단위로만 부여가 가능하며 개별 테이블에는 안 된다. employees.* (DB 전체) 또는 *.* (전체 DB) 형태만 허용하며 employees.department 같이 테이블을 지정할 수 없다. DB 권한도 글로벌 권한처럼 모든 DB에 적용할 수 있으므로 대상에 *.* 을 사용할 수 있다.

```
GRANT SELECT, INSERT ON mydb.* TO 'user'@'localhost' WITH GRANT OPTION;
```

GRANT OPTION 이 사용자가 자신이 부여받은 권한을 다른 사용자에게도 부여할 수 있는 권한이다.

비교

```
-- 1) user는 SELECT할 수 있지만 다른 사용자에게 SELECT 권한을 줄 수 없다
GRANT SELECT ON mydb.* TO 'user'@'localhost';
```

```
-- 2) user는 SELECT할 수 있고, 다른 사용자에게도 SELECT 권한을 줄 수 있다
GRANT SELECT ON mydb.* TO 'user'@'localhost' WITH GRANT OPTION;
```

테이블 권한

```
-- 1) 서버의 모든 DB에 대해 권한 부여
GRANT SELECT, INSERT, UPDATE, DELETE ON *.* TO 'user'@'localhost';

-- 2) 특정 DB의 모든 오브젝트에 대해 권한 부여
GRANT SELECT, INSERT, UPDATE, DELETE ON employees.* TO 'user'@'localhost';

-- 3) 특정 DB의 특정 테이블에 대해서만 권한 부여
GRANT SELECT, INSERT, UPDATE, DELETE ON employees.department TO
'user'@'localhost';
```

테이블 권한은 모든 DB에 대해 권한을 부여하는 것도 가능하며, 특정 DB의 오브젝트에 대해서만 권한을 부여하는 것도 가능하다. 또한 특정 DB의 특정 테이블에 대해서만 권한을 부여하는 것도 가능하다.

테이블의 특정 칼럼에 대해서만 권한을 부여하는 경우에는 GRANT 명령의 문법이 조금 달라져야 한다. 칼럼에 부여할 수 있는 권한은 DELETE를 제외한 INSERT, UPDATE, SELECT로 3가지이며, 각 권한 뒤에 칼럼을 명시하는 형태로 부여한다.

```
GRANT SELECT, INSERT, UPDATE(dept_name) ON employees.department TO
'user'@'localhost';
```

여러 가지 레벨이나 범위로 권한을 설정하는 것이 가능하지만 테이블이나 칼럼 단위의 권한은 잘 사용하지 않는다. 칼럼 단위의 권한이 하나라도 설정되면 나머지 모든 테이블의 모든 칼럼에 대해서도 권한 체크를 하기 때문에 칼럼 하나에 대해서만 권한을 설정하더라도 전체적인 성능에 영향을 미칠 수 있다. 꼭 필요하다면 GRANT 명령으로 해결하기보다는 테이블에서 권한을 허용하고자 하는 칼럼만으로 VIEW를 만들어 사용하는 방법도 고려해볼 수 있다. VIEW도 하나의 테이블로 인식되기 때문에 뷰를 만들어 두면 뷰의 칼럼에 대해 권한을 체크하지 않고 뷰 자체에 대한 권한만 체크하면 된다.

각 계정이나 권한에 부여된 권한이나 역할을 확인하기 위해서는 SHOW GRANTS 명령을 사용할 수도 있지만, 표 형태로 깔끔하게 보고자 한다면 mysql DB의 권한 관련 테이블을 참조하면 된다.

3.5 역할(Role)

MySQL 8.0부터 권한을 묶어서 역할(Role)로 사용할 수 있다. 실제 MySQL 서버 내부적으로 역할은 계정과 같은 모습이다. CREATE ROLE 명령을 이용해 역할을 정의할 수 있다.

역할 생성 및 권한 부여

예시

```
CREATE ROLE
    role_emp_read,
    role_emp_write;
```

이는 빈 껍데기만 있는 역할을 정의한 것이며, GRANT 명령으로 각 역할에 대해 실질적인 권한을 부여하면 된다.

```
-- role_emp_read 역할에는 employees DB의 모든 객체에 대해 읽기(SELECT) 권한만 부여
GRANT SELECT ON employees.* TO role_emp_read;

-- role_emp_write 역할에는 employees DB의 모든 객체에 대해 쓰기(INSERT, UPDATE,
DELETE) 권한을 부여
GRANT INSERT, UPDATE, DELETE ON employees.* TO role_emp_write;
```

기본적으로 역할은 그 자체로 사용될 수 없고 계정에 부여해야 한다. CREATE USER 명령으로 reader와 writer라는 계정을 생성해보자.

```
CREATE USER 'reader'@'127.0.0.1' IDENTIFIED BY 'qwerty';
CREATE USER 'writer'@'127.0.0.1' IDENTIFIED BY 'qwerty';
```

이제 GRANT 명령으로 reader와 writer 계정에 역할을 부여해보자.

```
GRANT role_emp_read TO 'reader'@'127.0.0.1';
GRANT role_emp_read, role_emp_write TO 'writer'@'127.0.0.1';
```

역할 활성화

지금 상태에서 reader나 writer 계정으로 로그인하면 employees DB의 데이터를 조회하거나 변경할 때 권한이 없다는 에러를 보게 된다. 실제 역할은 부여됐지만 아직 활성화되지 않았기 때문이다. SET ROLE 명령을 실행해 해당 역할을 활성화할 수 있다.

```
SET ROLE 'role_emp_read';
```

역할이 활성화되면 그 역할이 가진 권한은 사용할 수 있는 상태가 되지만, 계정이 로그아웃됐다가 다시 로그인하면 역할이 활성화되지 않은 상태로 초기화돼버린다. 이는 MySQL 서버의 역할이 자동으로 활성화되지 않게 설정돼 있기 때문이다.

사용자가 MySQL 서버에 로그인할 때 역할을 자동으로 활성화할지 여부를 activate_all_roles_on_login 시스템 변수로 설정할 수 있다. 다음과 같이 activate_all_roles_on_login 시스템 변수를 ON으로 설정하면 매번 SET ROLE 명령으로 역할을 활성화하지 않아도 로그인과 동시에 부여된 역할이 자동으로 활성화된다.

```
SET GLOBAL activate_all_roles_on_login=ON;
```

역할과 계정의 관계

여기까지 보면 MySQL 서버의 역할은 사용자 계정과 거의 비슷한 모습을 가지고 있다. 실제로 MySQL 서버 내부적으로도 역할과 계정은 동일한 객체로 취급된다. 단지 하나의 사용자 계정에 다른 사용자 계정이 가진 권한을 병합해서 권한 제어가 가능해졌을 뿐이다.

그럼 MySQL 서버는 계정과 역할을 어떻게 구분할까? 하나의 계정에 다른 계정의 권한을 병합하기만 하면 되므로 MySQL 서버는 역할과 계정을 구분할 필요가 없는 것이다.

mysql DB의 user 테이블을 살펴보면 실제 권한과 사용자 계정이 구분 없이 저장된 것을 확인할 수 있다.

```
SELECT user, host, account_locked FROM mysql.user;
```

역할을 생성할 때와 계정을 생성할 때의 차이점을 살펴보자.

일반적으로 CREATE USER 명령으로 계정을 생성할 때는 'reader'@'127.0.0.1'과 같이 계정 이름과 호스트 부분을 함께 명시한다. 하지만 CREATE ROLE 명령으로 역할을 생성할 때는 호스트 부분을 별도로 명시하지 않았다. 이것이 역할과 계정의 차이처럼 보일 수 있지만, 사실은 호스트 부분을 명시하지 않은 경우 자동으로 '모든 호스트(%)'가 자동으로 추가된다. 계정을 생성할 때도 'reader'라고만 계정 이름만 명시하면 'reader'@'%'와 동일한 계정이 생성된다.

역할과 계정 구분의 이유

역할과 계정은 내외부적으로 동일한 객체라고 했는데, MySQL 서버에서는 왜 굳이 CREATE ROLE 명령과 CREATE USER 명령을 구분해서 지원할까? 이는 데이터베이스 관리의 직무를 분리할 수 있게 해서 보안을 강화하는 용도로 사용될 수 있게 하기 위해서이다.

CREATE USER 명령에 대해서는 권한이 없지만 CREATE ROLE 명령만 실행 가능한 사용자는 역할을 생성할 수 있다. 이렇게 생성된 역할은 계정과 동일한 객체를 생성하지만, 실제 이 역할은 account_locked 칼럼의 값이 'Y'로 설정돼 있어서 로그인 용도로 사용할 수 없게 된다.

계정의 기본 역할 또는 역할에 부여된 역할 그래프 관계는 SHOW GRANTS 명령뿐만 아니라 mysql DB의 권한 관련 테이블을 통해서도 확인해볼 수 있다.

- mysql.default_roles
- mysql.role_edges

표 형태로 역할과 계정의 관계를 보고 싶다면 이 테이블들을 조회하면 된다.

요약

MySQL 8.0의 사용자 권한 관리 시스템은 다음과 같은 주요 특징을 가진다:

1. 계정 식별

- MySQL의 계정은 **사용자 ID + 접속 호스트**로 구성된다.
- 동일한 사용자 ID라도 접속 호스트가 다르면 별개의 계정으로 취급된다.
- 권한 충돌 시 범위가 좁은(구체적인) 계정이 우선 적용된다.

2. 계정 분류

- **시스템 계정:** SYSTEM_USER 권한을 가진 관리자 계정
- **일반 계정:** 응용 프로그램이나 개발자용 계정

- MySQL 8.0부터 이 구분이 명확해져 보안이 강화되었다.

3. 인증 방식

- **Native Authentication:** MySQL 5.7까지의 기본 방식
- **Caching SHA-2 Authentication:** MySQL 8.0의 기본 방식 (더 안전)
- 엔터프라이즈 에디션은 PAM, LDAP 등 외부 인증도 지원한다.

4. 비밀번호 관리

- **고수준 비밀번호:** validate_password 컴포넌트로 복잡도 검증
- **이중 비밀번호:** 서비스 중단 없이 비밀번호 변경 가능
- 유효기간, 재사용 금지 등 다양한 정책 설정 가능

5. 권한 체계

- **글로벌 권한:** DB나 테이블 외의 서버 전체 권한
- **객체 권한:** DB, 테이블, 칼럼 단위의 세부 권한
- MySQL 8.0부터 동적 권한 도입으로 더 세밀한 권한 관리 가능

6. 역할(Role)

- 권한을 그룹화하여 관리하는 기능
- 내부적으로는 계정과 동일한 객체
- 권한 관리의 효율성과 직무 분리를 통한 보안 강화

이러한 기능들을 통해 MySQL 8.0은 더욱 강력하고 유연한 사용자 권한 관리 시스템을 제공한다.